Chanakya University

Project for Laboratory Examination 2

Index Generation using Trie, Binary Search Tree and Python Dictionary

Course: Data Structures | Submitted to: Mr. Nikhil S. | Deadline: 2 PM, 14-10-2026

Student: Poorvith, B.Tech CSE (Second Year)

Contents

- 1. Problem statement in simple words
- 2. Design explanation
- 3. How to run the program
- 4. Test cases and results
- 5. Performance comparison (graphs and table)
- 6. Observations
- 7. Complete source code: index_generation.py, test_cases.py, benchmark.py

1. Problem statement in simple words

The task is to build an **index** of a text file, like the index at the back of a book. The program reads a file, finds every word, and writes an output file in which each distinct word appears once, in lexicographic (ASCII) order, followed by the line numbers on which it occurs.

```
apple : 1, 3
banana : 1, 2
cherry : 2
```

The index must be built in three ways: (a) a **Trie**, (b) a **Binary Search Tree**, (c) a **Python dictionary**. Each distinct word keeps its line numbers in a **linked-list queue**. The three versions are compared for time on six large files (more than one million words each) with 1000, 5000, 10000, 50000, 100000 and 500000 distinct words, for three actions: construction, searching 100000 random words, and printing the index in sorted order. The times are plotted with Matplotlib.

Rules followed: every printable ASCII character may be part of a word and words are compared in ASCII order; words are separated by one or more blanks, line breaks, full stops, commas, colons or semicolons; the program takes two command-line arguments (input file name, output file name).

2. Design explanation

Reading words. The file is read line by line. In each line the separators '.', ',', ':' and ';' are replaced by blanks, and the line is then split on blanks (line breaks and repeated blanks are ignored); all other characters stay inside the word.

Line-number queue. Each distinct word owns a singly linked list with a head and a tail pointer. A new line number is added at the tail in O(1), so the numbers stay in increasing order and can be read from the head. If a word occurs several times on the same line, that line number is stored only once.

Trie. Every node has a dictionary of children (character to node) and a field that holds the line queue if a word ends at that node. Insert and search take O(L) time for a word of length L, independent of how many

words are stored. Printing uses a recursive depth-first walk that visits children in ASCII order, so a word is printed before the longer words that extend it.

Binary Search Tree. Each node holds one distinct word, its line queue, and left and right children; smaller words go left and larger words go right (ASCII comparison). Insert and search take O(h) time, where h is the height: O(log n) on average and O(n) in the worst case. Printing uses an iterative in-order traversal, which gives sorted output.

Python dictionary. A hash table maps each word to its line queue. Insert and search take O(1) on average. Because a hash table has no order, the words are sorted (O(n log n)) when the index is printed.

Loops and recursion. BST insert, search and in-order printing use loops and an explicit stack, so a skewed tree cannot hit Python's recursion limit. The Trie printing function is recursive, but its depth is only the length of the longest word (the recursion limit is also raised in the program to be safe).

Same output. All three versions produce exactly the same text file; the test cases check this.

Structure	Insert / Search	Sorted printing	Extra memory
Trie	O(L)	O(total characters), no sorting	High (many nodes)
BST	O(log n) avg, O(n) worst	O(n), in-order walk	One node per word
Dictionary	O(1) average	O(n log n), needs sorting	Low

3. How to run the program

```
python index_generation.py input.txt output.txt          # dictionary version (default)
python index_generation.py input.txt output.txt trie     # Trie version
python index_generation.py input.txt output.txt bst      # BST version
python test_cases.py                                     # run all test cases
python benchmark.py                                      # full performance comparison + graph
python benchmark.py --small                              # quick test of the benchmark
```

The two required arguments are the input file name and the output file name. An optional third argument selects which of the three versions builds the index. Matplotlib is needed for benchmark.py.

4. Test cases and results

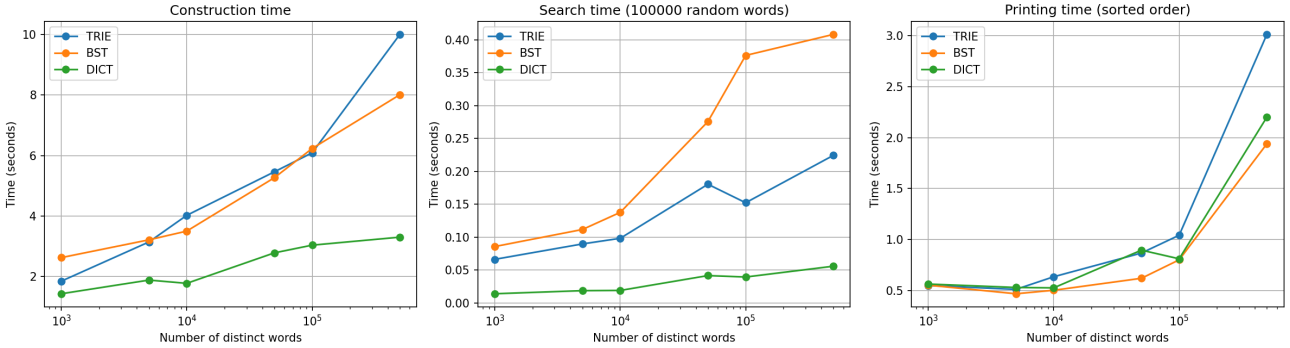
Each test builds the index with the Trie, the BST and the dictionary, checks that all three outputs are identical, and compares with the expected text. All tests passed.

Test	Input idea	What it checks
test_basic_and_repeats	apple banana apple / cherry banana / apple	Repeated words, line numbers 1, 3 for apple
test_all_separators	a,b.c:d;e f,,..;; g	Every separator and runs of separators
test_ascii_order_and_prefix	b B 2 a ab abc A	ASCII order: digits, capitals, lowercase; prefix before longer w
test_special_characters	!hi #tag a-b x_y ~z	Printable ASCII symbols inside words
test_blank_lines_and_line_numbers	blank lines, then zeta / zeta alpha	Correct line numbers when lines are empty
test_empty_file	(empty file)	Empty input gives empty output

```
PASSED: test_basic_and_repeats
PASSED: test_all_separators
PASSED: test_ascii_order_and_prefix
PASSED: test_special_characters
PASSED: test_blank_lines_and_line_numbers
PASSED: test_empty_file
All tests passed.
```

5. Performance comparison

Six input files were generated, each with 1,100,000 words and 1000, 5000, 10000, 50000, 100000 and 500000 distinct words (random words made of printable ASCII characters). For each file the three structures were timed for (a) building the complete index with linked-list queues, (b) searching 100000 random words (half present, half mostly absent), and (c) printing the index in lexicographic order to a text file. Times are in seconds.



Measured times (seconds)

Distinct words	Trie build	BST build	Dict build	Trie search	BST search	Dict search	Trie print	BST print	Dict print
1000	1.84	2.62	1.43	0.07	0.09	0.01	0.55	0.55	0.56
5000	3.14	3.21	1.88	0.09	0.11	0.02	0.51	0.47	0.53
10000	4.02	3.50	1.77	0.10	0.14	0.02	0.64	0.50	0.53
50000	5.46	5.27	2.78	0.18	0.28	0.04	0.87	0.62	0.90
100000	6.09	6.23	3.03	0.15	0.38	0.04	1.04	0.80	0.81
500000	10.01	8.01	3.30	0.22	0.41	0.06	3.01	1.94	2.20

6. Observations

- **Construction:** the dictionary is fastest for every size (about 1.4 s to 3.3 s) and grows slowly. The Trie and the BST are slower and grow with the number of distinct words. They are close to each other up to 100000 words (the Trie wins at some sizes, the BST at others); at 500000 words the BST (8.0 s) is faster than the Trie (10.0 s).
- **Search:** the dictionary is fastest (0.01 s to 0.06 s for 100000 searches). The Trie is second at every size and the BST is slowest, because a BST search makes about log n string comparisons while a Trie search depends on word length only.
- **Printing:** the three are within about 0.3 s of each other up to 100000 words. At 500000 words the BST is fastest (1.94 s), then the dictionary (2.20 s), and the Trie is slowest (3.01 s) because its traversal visits millions of nodes and builds prefix strings. Part of the printing time is writing the line numbers, which is the same for all three.
- **Overall:** for this task the dictionary is the best in building and searching. The Trie gives predictable O(L) operations and gives sorted output without a sort, but costs more memory and time in Python. The BST is simple and prints sorted words directly, but its search slows as the tree grows. The BST can degrade to O(n) if words arrive in sorted order; the random input used here keeps it balanced on average.
- The times were measured on one machine and vary a little from run to run, and they will differ on another computer, but the trends should be similar.

7. Complete source code

7.1 index_generation.py

```
"""
Index Generation Utility (Lab Exam 2, Data Structures)

Usage:  python index_generation.py <input_file> <output_file> [trie|bst|dict]

Reads a text file and writes an index: every distinct word (in ASCII order)
followed by the line numbers where it appears.

Design in short:
  * Separators between words: blank, line break, '.', ',', ':', ';'
    Every other printable ASCII character is part of a word.
  * Every distinct word keeps its line numbers in a linked-list queue.
  * Three ways to store the words: Trie, BST, Python dict.
"""

import sys

sys.setrecursionlimit(100000)   # the Trie printing is recursive (depth = word length)


# ---------------------------------------------------------------
# Linked-list queue of line numbers
# ---------------------------------------------------------------
class LineNode:
    def __init__(self, line_no):
        self.line_no = line_no
        self.next = None


class LineQueue:
    def __init__(self):
        self.head = None
        self.tail = None

    def enqueue(self, line_no):
        """Add a line number at the tail."""
        if self.tail is not None and self.tail.line_no == line_no:
            return                      # same word twice on one line: store once
        node = LineNode(line_no)
        if self.head is None:           # queue was empty
            self.head = node
        else:
            self.tail.next = node
        self.tail = node

    def to_list(self):
        """Read all line numbers from head to tail."""
        numbers = []
        cur = self.head
        while cur is not None:
            numbers.append(cur.line_no)
            cur = cur.next
        return numbers


# ---------------------------------------------------------------
# 1) Trie
# ---------------------------------------------------------------
class TrieNode:
    def __init__(self):
        self.children = {}      # character -> TrieNode
        self.lines = None       # LineQueue if a word ends at this node


class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word, line_no):
        node = self.root
        for ch in word:                         # walk down, creating nodes
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
```

```
        if node.lines is None:                  # first time we see this word
            node.lines = LineQueue()
        node.lines.enqueue(line_no)

    def search(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node.lines                       # None if no word ends here

    def items(self):
        """All (word, LineQueue) pairs in ASCII order."""
        result = []
        self._collect(self.root, "", result)
        return result

    def _collect(self, node, prefix, result):
        if node.lines is not None:              # a word ends here (comes first)
            result.append((prefix, node.lines))
        for ch in sorted(node.children):        # smallest character first
            self._collect(node.children[ch], prefix + ch, result)


# ---------------------------------------------------------------
# 2) Binary Search Tree
# ---------------------------------------------------------------
class BSTNode:
    def __init__(self, word):
        self.word = word
        self.lines = LineQueue()
        self.left = None
        self.right = None


class BST:
    def __init__(self):
        self.root = None

    def insert(self, word, line_no):
        # go down the tree until we find the word or reach an empty place
        parent = None
        cur = self.root
        while cur is not None and cur.word != word:
            parent = cur
            cur = cur.left if word < cur.word else cur.right
        if cur is None:                         # word is new: attach a new node
            cur = BSTNode(word)
            if parent is None:
                self.root = cur
            elif word < parent.word:
                parent.left = cur
            else:
                parent.right = cur
        cur.lines.enqueue(line_no)

    def search(self, word):
        cur = self.root
        while cur is not None and cur.word != word:
            cur = cur.left if word < cur.word else cur.right
        if cur is None:
            return None
        return cur.lines

    def items(self):
        """All (word, LineQueue) pairs in ASCII order (in-order traversal)."""
        result = []
        stack = []
        cur = self.root
        while cur is not None or stack:
            while cur is not None:              # go as far left as possible
                stack.append(cur)
                cur = cur.left
            cur = stack.pop()
            result.append((cur.word, cur.lines))
            cur = cur.right
        return result
# ---------------------------------------------------------------
# 3) Python dictionary
```

```
# ---------------------------------------------------------------
class DictIndex:
    def __init__(self):
        self.table = {}                         # word -> LineQueue

    def insert(self, word, line_no):
        if word not in self.table:
            self.table[word] = LineQueue()
        self.table[word].enqueue(line_no)

    def search(self, word):
        return self.table.get(word)

    def items(self):
        # sorted() on strings uses ASCII order
        return [(word, self.table[word]) for word in sorted(self.table)]


# ---------------------------------------------------------------
# Reading the file and writing the index
# ---------------------------------------------------------------
STRUCTURES = {"trie": Trie, "bst": BST, "dict": DictIndex}


def build_index(structure_name, input_file):
    index = STRUCTURES[structure_name]()
    with open(input_file, "r", encoding="ascii", errors="replace") as f:
        for line_no, line in enumerate(f, start=1):
            for separator in ".,:;":            # turn separators into blanks
                line = line.replace(separator, " ")
            for word in line.split():           # split() ignores extra blanks
                index.insert(word, line_no)
    return index


def write_index(index, output_file):
    with open(output_file, "w", encoding="ascii", errors="replace") as out:
        for word, queue in index.items():
            numbers = ", ".join(str(n) for n in queue.to_list())
            out.write(word + " : " + numbers + "\n")


def main():
    if len(sys.argv) < 3:
        print("Usage: python index_generation.py <input_file> <output_file> [trie|bst|dict]")
        sys.exit(1)
    input_file = sys.argv[1]
    output_file = sys.argv[2]
    method = sys.argv[3].lower() if len(sys.argv) > 3 else "dict"
    if method not in STRUCTURES:
        print("Method must be trie, bst or dict")
        sys.exit(1)

    index = build_index(method, input_file)
    write_index(index, output_file)
    print("Index built using", method, "- written to", output_file)


if __name__ == "__main__":
    main()
```

7.2 test_cases.py

```
"""
Test cases for the index generator.
Run:  python test_cases.py
"""

import os
import tempfile

from index_generation import STRUCTURES, build_index, write_index


def run(text):
    """Build the index with all 3 structures; return their outputs (must match)."""
    outputs = []
    with tempfile.TemporaryDirectory() as d:
        inp = os.path.join(d, "in.txt")
        with open(inp, "w") as f:
            f.write(text)
        for name in STRUCTURES:
            out = os.path.join(d, name + ".txt")
            write_index(build_index(name, inp), out)
            with open(out) as f:
                outputs.append(f.read())
    assert outputs[0] == outputs[1] == outputs[2], "Trie/BST/dict outputs differ!"
    return outputs[0]


def test_basic_and_repeats():
    out = run("apple banana apple\ncherry banana\napple\n")
    assert out == "apple : 1, 3\nbanana : 1, 2\ncherry : 2\n"


def test_all_separators():
    # blank, comma, full stop, colon, semicolon, multiple together
    out = run("a,b.c:d;e   f,,..;;  g\n")
    assert out == "a : 1\nb : 1\nc : 1\nd : 1\ne : 1\nf : 1\ng : 1\n"


def test_ascii_order_and_prefix():
    # ASCII: digits < uppercase < lowercase; a prefix comes before its extensions
    out = run("b B 2 a ab abc A\n")
    assert out == "2 : 1\nA : 1\nB : 1\na : 1\nab : 1\nabc : 1\nb : 1\n"


def test_special_characters():
    out = run("!hi #tag a-b x_y ~z\n")
    words = [line.split(" : ")[0] for line in out.splitlines()]
    assert words == sorted(words)
    assert "!hi" in words and "a-b" in words


def test_blank_lines_and_line_numbers():
    out = run("\n\nzeta\n\nzeta alpha\n")
    assert out == "alpha : 5\nzeta : 3, 5\n"


def test_empty_file():
    assert run("") == ""


if __name__ == "__main__":
    for name, fn in list(globals().items()):
        if name.startswith("test_"):
            fn()
            print("PASSED:", name)
    print("All tests passed.")
```

7.3 benchmark.py

```
"""
Benchmark for Trie vs BST vs dict index (Lab Exam 2).

Usage:
    python benchmark.py            # full run: 1,100,000 words per file
    python benchmark.py --small    # quick test run: 20,000 words per file
For each distinct-word count it:
```

```
  1. generates a text file (more than 1 million words)
  2. times (a) construction, (b) 100000 random searches, (c) printing
  3. finally draws three Matplotlib graphs (saved as PNG).
"""

import os
import random
import string
import sys
import time

import matplotlib
matplotlib.use("Agg")            # draw to file, no window needed
import matplotlib.pyplot as plt

from index_generation import STRUCTURES, build_index, write_index

if "--small" in sys.argv:
    TOTAL_WORDS = 20_000
    DISTINCT_COUNTS = [100, 500, 1000, 5000, 10000, 15000]
    NUM_SEARCHES = 5_000
else:
    TOTAL_WORDS = 1_100_000   # more than a million, as the PDF requires
    DISTINCT_COUNTS = [1000, 5000, 10000, 50000, 100000, 500000]
    NUM_SEARCHES = 100_000

WORDS_PER_LINE = 10
# Letters, digits and some punctuation (none of them are separators)
ALPHABET = string.ascii_letters + string.digits + "!#$%&()*+-/<=>?@[]^_{|}~"


def make_distinct_words(count):
    words = set()
    while len(words) < count:
        length = random.randint(3, 10)
        words.add("".join(random.choice(ALPHABET) for _ in range(length)))
    return list(words)


def generate_file(filename, distinct):
    vocab = make_distinct_words(distinct)
    # every distinct word at least once, rest are random repeats
    all_words = vocab + random.choices(vocab, k=TOTAL_WORDS - distinct)
    random.shuffle(all_words)
    with open(filename, "w") as f:
        for i in range(0, len(all_words), WORDS_PER_LINE):
            f.write(" ".join(all_words[i:i + WORDS_PER_LINE]) + "\n")
    return vocab


def main():
    random.seed(1)
    results = {name: {"build": [], "search": [], "print": []} for name in STRUCTURES}

    for distinct in DISTINCT_COUNTS:
        filename = f"input_{distinct}.txt"
        print(f"\n=== {distinct} distinct words ===")
        vocab = generate_file(filename, distinct)
        # half the searches are words that exist, half are new random strings
        # (most of those will not be in the file), so both outcomes are timed
        queries = random.choices(vocab, k=NUM_SEARCHES // 2)
        queries += ["".join(random.choice(ALPHABET) for _ in range(random.randint(3, 10)))
                    for _ in range(NUM_SEARCHES - NUM_SEARCHES // 2)]
        random.shuffle(queries)

        for name in STRUCTURES:
            # (a) construction
            t = time.perf_counter()
            index = build_index(name, filename)
            build_t = time.perf_counter() - t

            # (b) search
            t = time.perf_counter()
            for w in queries:
                index.search(w)
            search_t = time.perf_counter() - t

            # (c) printing in sorted order
            t = time.perf_counter()
            write_index(index, f"output_{name}_{distinct}.txt")
```

```
            print_t = time.perf_counter() - t

            results[name]["build"].append(build_t)
            results[name]["search"].append(search_t)
            results[name]["print"].append(print_t)
            print(f"{name:5s} build={build_t:7.2f}s  search={search_t:6.2f}s  print={print_t:6.2f}s")

        os.remove(filename)   # input files are huge; delete after use

    # ---- plotting ----
    titles = {"build": "Construction time", "search": f"Search time ({NUM_SEARCHES} random words)",
              "print": "Printing time (sorted order)"}
    fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
    for ax, action in zip(axes, ["build", "search", "print"]):
        for name in STRUCTURES:
            ax.plot(DISTINCT_COUNTS, results[name][action], marker="o", label=name.upper())
        ax.set_title(titles[action])
        ax.set_xlabel("Number of distinct words")
        ax.set_ylabel("Time (seconds)")
        ax.set_xscale("log")
        ax.grid(True)
        ax.legend()
    plt.tight_layout()
    plt.savefig("performance_comparison.png", dpi=150)
    print("\nSaved graph: performance_comparison.png")


if __name__ == "__main__":
    main()
```